6061 | Introduction to Deep Learning

Exercise 4

Prof. Dr. Florian Lemmerich[1] Andreas Einwiller[1] Max Klabunde[1*]

[1]Professorship for Applied Machine Learning
University of Passau

[*](2021–2025) Preparation, creation, and presentation

12.11.2026

Goals

- Use PyTorch Dataset and DataLoader to implement stochastic gradient descent (SGD)
- Understand how PyTorch computes gradients
- Perform automatic differentiation (backpropagation) by hand

Basic Gradient Descent in PyTorch

```
loss_func = torch.nn.MSELoss()  # default: reduction='mean'
for epoch in range(n_epochs):
    output = model(x)
    loss = loss_func(output, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Section 1

Stochastic Gradient Descent

Gradient Descent

1. Compute the gradient at the current point
2. Take a small step in the direction of the negative gradient
3. Go to 1.

Basic Gradient Descent in PyTorch

```
loss_func = torch.nn.MSELoss()  # default: reduction='mean'
for epoch in range(n_epochs):
    output = model(x)
    loss = loss_func(output, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

What if `x` is a very large tensor?

Stochastic Gradient Descent (SGD)

Approximate the gradient using a subset of the data:

1. Shuffle the dataset and partition it into batches of b samples
2. For each batch:
    1. Calculate gradients at current point
    2. Take a small step against the gradient
3. Go to 1.

Each update is cheaper, and gradient noise can help escape saddle points and sharp minima. This often improves generalization

Stochastic Gradient Descent

Update Formula

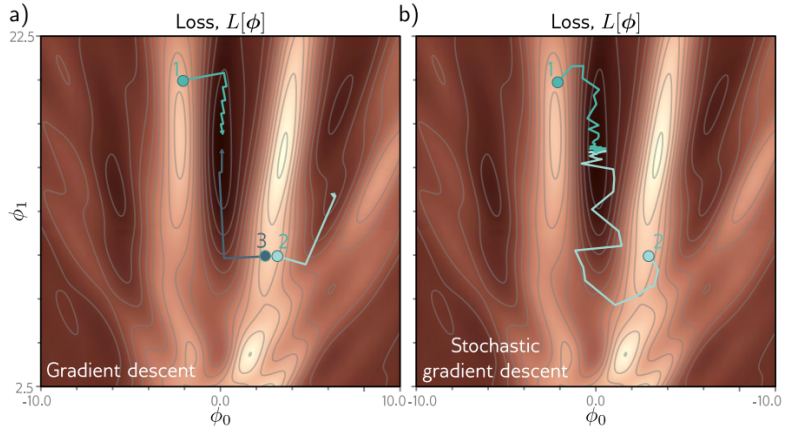
Full-batch Gradient Descent

$$w_{t+1} = w_t - \alpha \cdot \frac{1}{n} \sum_{i=1}^{n} \frac{\partial \ell(w_t, x_i, y_i)}{\partial w}$$

Stochastic Gradient Descent

$$w_{t+1} = w_t - \alpha \cdot \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \frac{\partial \ell(w_t, x_i, y_i)}{\partial w}$$

Stochastic Gradient Descent visualized



Simon J.D. Prince, Understanding Deep Learning. Fig. 6.5

SGD in PyTorch

1. Wrap the data in a `torch.utils.data.Dataset`
2. Pass the dataset to a `torch.utils.data.DataLoader`
3. Iterate over the DataLoader to get batches
4. Train as usual

$\Rightarrow$ Demo

How to choose batch size?

- Too small: updates are noisy
- Too large: each update is expensive[1]
- Typically: as large as fits into GPU memory
- Commonly powers of 2, but not strictly necessary
  - For maximum performance (to use tensor cores with mixed precision training), the batch size should be a multiple of 8[2]
- Treat batch size as a hyperparameter (alongside α, ...)

[1] Keskar et al. (2016) show that solutions found with large batch sizes (> 512) tend to generalize worse.

[2] NVIDIA cuDNN Docs

Section 2

Automatic Differentiation

How does PyTorch compute gradients?

Blackboard

Feedback

Please use the text field



https://lgms.nl/p/presentation-feedback/?code=119ab3